

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_HIST 5           // Capacity limit for history buffer
#define MAX_ARGS 32
#define MAX_CMDS 10
#define BUF_SIZE 512

// Structure for Circular History Buffer
typedef struct {
    char *entries[MAX_HIST];
    int start;                // Index of oldest entry
    int count;                // Current total count
    int next_id;              // Sequential global ID
} HistoryBuffer;

HistoryBuffer hist = { .start = 0, .count = 0, .next_id = 1 };

// 1. HISTORY BUFFER MANAGEMENT
void add_history(const char *cmd) {
    if (strlen(cmd) == 0) return;

    // Handle capacity limit using ring buffer eviction
    if (hist.count == MAX_HIST) {
        free(hist.entries[hist.start]); // Evict oldest
        hist.entries[hist.start] = strdup(cmd);
        hist.start = (hist.start + 1) % MAX_HIST;
    } else {
        int idx = (hist.start + hist.count) % MAX_HIST;
        hist.entries[idx] = strdup(cmd);
        hist.count++;
    }
    hist.next_id++;
}
```

```

void display_history() {
    printf("--- Command History (Max Capacity: %d) ---\n", MAX_HIST);
    for (int i = 0; i < hist.count; i++) {
        int idx = (hist.start + i) % MAX_HIST;
        int display_id = hist.next_id - hist.count + i;
        printf(" %d  %s\n", display_id, hist.entries[idx]);
    }
}

void cleanup_history() {
    for (int i = 0; i < hist.count; i++) {
        int idx = (hist.start + i) % MAX_HIST;
        free(hist.entries[idx]);
    }
    hist.count = 0;
}

// 2. PIPELINE SYSTEM IMPLEMENTATION
void execute_pipeline(char *cmd_str) {
    char *cmds[MAX_CMDS];
    int num_cmds = 0;

    // Parse commands separated by '|'
    char *token = strtok(cmd_str, "|");
    while (token != NULL && num_cmds < MAX_CMDS) {
        cmds[num_cmds++] = token;
        token = strtok(NULL, "|");
    }

    if (num_cmds == 0) return;

    int pipefds[2 * (num_cmds - 1)];
    for (int i = 0; i < num_cmds - 1; i++) {
        if (pipe(pipefds + i * 2) < 0) {
            perror("Pipe creation failed");
            return;
        }
    }
}

```

```

    }
}

for (int i = 0; i < num_cmds; i++) {
    // Tokenize command into args
    char *args[MAX_ARGS];
    int arg_count = 0;
    char *arg_token = strtok(cmds[i], " \t");
    while (arg_token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = arg_token;
        arg_token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;

    if (arg_count == 0) continue;

    pid_t pid = fork();
    if (pid == 0) {
        // Child Process Wiring
        if (i != 0) {
            // Not first command: redirect STDIN from previous pipe
            dup2(pipefds[(i - 1) * 2], STDIN_FILENO);
        }
        if (i != num_cmds - 1) {
            // Not last command: redirect STDOUT to next pipe
            dup2(pipefds[i * 2 + 1], STDOUT_FILENO);
        }

        // Close all pipe file descriptors in child
        for (int j = 0; j < 2 * (num_cmds - 1); j++) {
            close(pipefds[j]);
        }

        execvp(args[0], args);
        perror("Execution failed");
        exit(EXIT_FAILURE);
    }
}

```

```

    }

    // Parent closes all pipe file descriptors
    for (int i = 0; i < 2 * (num_cmds - 1); i++) {
        close(pipefds[i]);
    }

    // Wait for all child processes to terminate
    for (int i = 0; i < num_cmds; i++) {
        wait(NULL);
    }
}

int main() {
    char input_buf[BUF_SIZE];
    char input_copy[BUF_SIZE];

    while (1) {
        printf("\nPipeline Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        // Copy input for history and parsing
        strcpy(input_copy, input_buf);
        add_history(input_copy);

        if (strcmp(input_buf, "history") == 0) {
            display_history();
        } else {
            execute_pipeline(input_buf);
        }
    }

    cleanup_history();
    printf("Shell exited safely.\n");
    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano history_pipeline.c
abhis@MIKHAIL-37:~$ gcc -g -Wall history_pipeline.c -o history_pipeline
abhis@MIKHAIL-37:~$ ./history_pipeline
```

```
Pipeline Shell > ls -l | grep history
-rwxr-xr-x 1 abhis abhis 22144 Sep 29 09:58 history_pipeline
-rw-r--r-- 1 abhis abhis  4223 Sep 29 09:58 history_pipeline.c
-rwxr-xr-x 1 abhis abhis 16192 Sep 16 06:50 history_shell
-rw-r--r-- 1 abhis abhis  1129 Sep 16 06:48 history_shell.c
-rwxr-xr-x 1 abhis abhis 21288 Sep 29 03:32 shell_history
-rw-r--r-- 1 abhis abhis  3394 Sep 29 03:41 shell_history.c
```

```
Pipeline Shell > cat /etc/passwd | cut -d: -f1 | sort
_apt
_chrony
abhis
backup
bin
daemon
dhcpcd
games
irc
landscape
list
lp
mail
man
messagebus
news
nobody
polkitd
proxy
root
sync
sys
syslog
systemd-network
systemd-resolve
uucp
www-data
```

```
Pipeline Shell > exit
Shell exited safely.
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./history_pipeline
==1646== Memcheck, a memory error detector
==1646== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1646== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1646== Command: ./history_pipeline
==1646==

Pipeline Shell > exit
Shell exited safely.
==1646==
==1646== HEAP SUMMARY:
==1646==      in use at exit: 0 bytes in 0 blocks
==1646==    total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1646==
==1646== All heap blocks were freed -- no leaks are possible
==1646==
==1646== For lists of detected and suppressed errors, rerun with: -s
==1646== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```